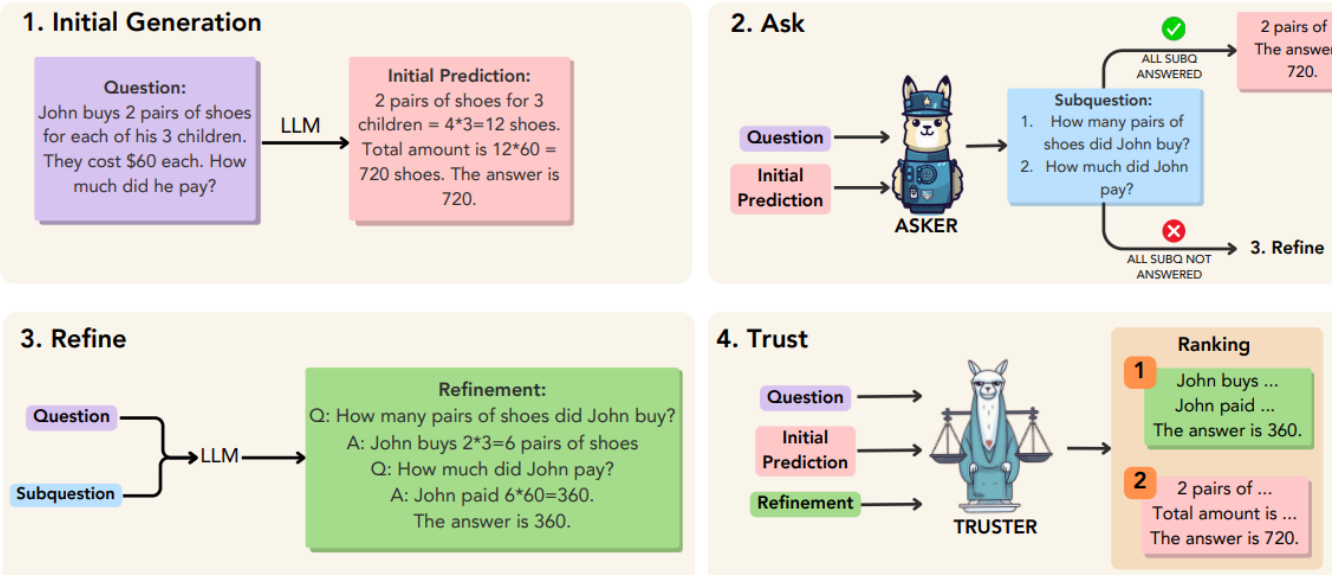
# The ART of LLM Refinement: Ask, Refine, and Trust

> #Inference #Finetuning #CoT




- What problems does this framework solve?
  - Though LLMs have solid ability to generate and answer the questions, they are not able to judge the quality in some complicated tasks, such as math problem solving.
  - The current self-refinement is (was) based on the assumption that models can find out the mistakes and effectively fix them. However, it is observed that this is not the case. Sometimes models will "fix" a correct answer to the wrong one.
- What does this framework do?
  - This framework introduces three steps to help LLMs to better find, understand, and fix the errors in the response: *ASK*, *REFINE*, and *TRUST*
  - Firstly, we let an LLM generate an initial response, given a prompt $x$.
  - **ASK**
    - After the LLM generates the initial response, we let the Asker model $\phi$ to come up with some sub-questions that are necessary to correctly answer prompt $x$.
    - Then, with the initial response, the Asker will check whether the sub-questions are correctly answered. Mark `No` if all sub-questions are answered (no need for refinement), and `Yes` otherwise.
    - The Asker model $\phi$ is finetuned by 1. Current-task related dataset with CoT and 2. Sub-question generation and "YES-NO" judge and labeling dataset.
  - **Refine**
    - If the Asker returns `Yes`, then use the original input $x$ and the sub-questions generated by the Asker as the input for the LLM to generate another version of answer.
  - **Trust**
    - The Truster model $\theta$ is a model that will distinguish between the initial response and the

refined version. We need the Truster model to be able to tell why an answer is better than the other one by finetuning it.
    - To achieve this, we prepare a dataset that contains a good answer and a bad answer $(y_j, y_k)$, where $y_j$ is the preferred answer and $y_k$ is the wrong answer. We calculate the score difference between them, and use the difference as the objective. If the score of $y_j$ is higher than that of $y_k$, then the loss (negative of the difference) will be smaller.

      $$L(\theta) = -\mathbb{E}_{(x,y_j,y_k)\sim D}\left[\log \sigma\left(r_\theta(x, y_j) - r_\theta(x, y_k)\right)\right]$$

- Experiment Results
  - Performance Enhancement: ART significantly outperforms the standard Self-Refinement baseline.
    - On GSM8K, using LLaMA-7B as an Asker to guide ChatGPT yields a 2 percentage point improvement over ChatGPT's self-refinement.
    - When combined with the Trust module, performance further increases, even surpassing the results of fine-tuning large models.
  - Key Findings:
    - Don't Always Refine: Experiments show that refining all samples (100% refinement) leads to a significant drop in accuracy. The best results come from refining only about 30-35% of difficult samples selected by the Asker.
    - Small Models Can Handle It: A fine-tuned 7B model is more accurate than GPT-4 or 70B models at determining “when to refine.”
- My Thought
  - Although this is an older project, its design remains instructive for today's design of multi-agent + tool-calling frameworks.
  - Instead of a single-way inference of generating the steps of tool-calling (sub-questions), we can let the planner run a single-way inference and introduce another agent that can verify whether the sub-questions can handle the task or not. If refinement is needed, then return a flag (yes-no) and add the suggestion, and feedback to the planner to refine the answer.
  - Then, we can have a truster to validate whether the new tool-call trajectory is better than the old one.
  - This multi-factor authentication mechanism can ensure the quality of task handling (this mechanism can be integrated with the Yunjue Agent Framework.
- Work Cited
  - Paper
  - Chat History
  - NotebookML